

FOODRESCUE

Kế hoạch thực hiện chi tiết từ đầu đến cuối · nhóm 4 người

8 tuần · 5 giai đoạn · mỗi giai đoạn kết thúc bằng một **cổng kiểm tra** phải đạt mới đi tiếp

Mỗi việc ghi rõ: **làm các bước nào, theo thứ tự nào, cần gì trước, xong thì bàn giao gì, kiểm tra bằng cách nào**

Dựa trên: FoodRescue_TaiLieuDuAn.pdf (v4.4) · foodrescue_schema.sql (v5.4) · FoodRescue_QuyUocChung.pdf · thư mục quy_uoc_chung/

Phiên bản 3 · Ngày 26/09/2026

Mục lục

1. Cách đọc kế hoạch này

1.1 Ba nguyên tắc · 1.2 Mỗi thẻ việc gồm · 1.3 Bốn module · 1.4 Lịch tuần

2. Lộ trình 5 giai đoạn

3. Làm gì trước, làm gì sau: từng giai đoạn

GD0 Chuẩn bị · GD1 Nền tảng và hợp đồng API · GD2 Chức năng lõi · GD3 Nhận hàng, bán đồ, quản lý túi · GD4 Tiền và quản trị · GD5 Hoàn thiện và bàn giao (kèm cổng kiểm tra)

4. Thẻ việc chi tiết

TV1 · TV2 · TV3 · TV4

5. Phân chia thư mục và điểm nối

5.1 Ai sở hữu thư mục nào · 5.2 Điểm nối giữa các module · 5.3 Quy ước JSON chung

6. Quy trình làm việc với Git

7. Hợp nhóm, theo dõi tiến độ, Definition of Done

1. Cách đọc kế hoạch này

1.1 Ba nguyên tắc

1. **Chia theo module đọc:** mỗi người làm cả backend lẫn frontend cho phần của mình (mục 1.3). Không ai phải chờ người khác viết API hay giao diện cho mình.
2. **Làm theo giai đoạn có cổng kiểm tra:** hết mỗi giai đoạn cả nhóm cùng chạy thử trên nhánh `develop`. Đạt đủ điều kiện của cổng mới sang giai đoạn sau; chưa đạt thì dồn người vào phần đang trễ (mục 7.4).
3. **Trong một giai đoạn, làm theo thứ tự "hợp đồng → backend → frontend":** viết DTO và API giả trước (để người khác dùng được ngay), rồi viết phần xử lý thật, cuối cùng nối giao diện.

1.2 Mỗi thẻ việc gồm

Cột	Ý nghĩa
Mã, Tuần	Mã để đặt tên nhánh Git và thẻ Trello; tuần (và ngày, với tuần 1) phải xong
Các bước làm	Làm lần lượt từ trên xuống
Cần có trước	Việc phải xong trước; "seed" = chỉ cần dữ liệu mẫu trong SQL
Kết quả bàn giao	Thứ nhìn thấy / chạy được khi việc xong
Kiểm tra	Test case (TC) trong tài liệu dự án mục 13, câu kiểm tra (C) trong <code>foodrescue_checks.sql</code> , hoặc cách thử bằng tay

1.3 Bốn module

Người	Phụ trách (backend + frontend)	Giờ
TV1 · Nền tảng, Tài khoản, Tiền (nhóm trưởng)	Repo, khung dự án, entity, bảo mật JWT, 3 hàm dùng chung (sổ kho, nhật ký, hoàn tiền); đăng nhập, hồ sơ; admin: đơn hàng, hoàn tiền, chi trả theo kỳ, đối soát, nhật ký; deploy, tổng hợp báo cáo	~99
TV2 · Cửa hàng, Túi, Kho	Đăng ký cửa hàng, admin duyệt và đặt phí; đăng / sửa / huỷ túi, thêm / bớt túi, sổ kho; hồ sơ cửa hàng; nhân viên; admin quản lý người dùng; doanh thu, sao kê, yêu cầu rút tiền	~90
TV3 · Khách hàng, Bản đồ, Giao diện chung	Bộ component giao diện dùng chung (phong cách mẫu Capichi); trang chủ, tìm kiếm, khoảng cách OSRM, bản đồ, chi tiết túi, trang cửa hàng; đánh giá; admin danh mục và thống kê chung; dữ liệu demo	~90
TV4 · Giỏ hàng, Đơn, Thanh toán, Giao hàng	Giỏ hàng, tạo đơn nhiều túi và chống bán vượt; VNPay Sandbox; đơn của tôi, mã nhận hàng; màn hình quầy, giao hàng bằng mã; đơn của cửa hàng, huỷ đơn; tác vụ tự động	~86

Giờ TV1 gồm khoảng 16 giờ việc nhóm trưởng và tổng hợp báo cáo. Khối lượng khoảng 11-12 giờ mỗi người mỗi tuần.

1.4 Lịch tuần (ví dụ bắt đầu thứ Hai 28/09/2026)

T1	T2	T3	T4	T5	T6	T7	T8
28/09-04/10	05/10-11/10	12/10-18/10	19/10-25/10	26/10-01/11	02/11-08/11	09/11-15/11	16/11-22/11

2. Lộ trình 5 giai đoạn

Giai đoạn	Thời gian	Mục tiêu	Kết quả khi qua cổng (demo được gì)	Tag Git
GĐ0 Chuẩn bị	Tuần 1, ngày 1-2	Máy ai cũng chạy được CSDL và khung dự án; repo sẵn sàng	<code>git clone</code> → backend chạy cổng 8080, Swagger mở được; frontend chạy cổng 5173 thấy 3 khu (khách, cửa hàng, admin); CSDL v5.4 chạy, <code>foodrescue_checks.sql</code> sạch trên mọi máy	—
GĐ1 Nền tảng và hợp đồng API	Tuần 1, ngày 3-5	Có đủ "đồ dùng chung" và hợp đồng API để 4 người code song song	13 entity khớp CSDL; JWT + DevAuth chạy; 3 hàm dùng chung có test; Swagger hiện toàn bộ API (trả dữ liệu giả); trang chủ tĩnh đúng phong cách mẫu; mỗi người có giao diện tĩnh của phần mình	v0.1
GĐ2 Chức năng lõi	Tuần 2-3	Có hàng để bán, khách tìm thấy, thêm vào giỏ, đặt và thanh toán được	Đăng ký cửa hàng → admin duyệt → khách thấy túi trên trang chủ kèm khoảng cách → thêm vào giỏ → đặt → trả bằng thẻ test VNPAY → đơn "Chờ nhận hàng". Không bán vượt khi 2 người cùng đặt	v0.3
GĐ3 Nhận hàng, bản đồ, quản lý túi	Tuần 4-5	Trộn luồng mua hàng đến lúc giao; cửa hàng tự quản lý túi và kho	Kịch bản demo mục 14.2 tài liệu dự án chạy được bước 1-6: bản đồ, chi tiết túi, đơn của tôi + QR, màn hình quầy, nhập mã giao hàng; chủ cửa hàng đăng / sửa / huỷ túi, thêm / bớt túi, xem sổ kho; đơn tự hết hạn, tự ghi "không đến lấy"	v0.5
GĐ4 Tiền và quản trị	Tuần 6-7	Đủ mọi chức năng trong tài liệu	Huỷ đơn → hoàn tiền; chi trả theo kỳ và cửa hàng yêu cầu rút tiền; đối soát chênh lệch 0; nhật ký; nhân viên; doanh thu và sao kê; đánh giá; admin người dùng, danh mục, thống kê. Chạy đủ 26 test case	v0.9
GĐ5 Hoàn thiện và bàn giao	Tuần 8	Không còn lỗi nghiêm trọng; có báo cáo, slide, demo	Test chéo xong, lỗi về tiền / kho / phân quyền đã sửa; <code>foodrescue_checks.sql</code> sạch sau toàn bộ test; (tuỳ chọn) chạy online; báo cáo + slide + video demo	v1.0

Việc	T1	T2	T3	T4	T5	T6	T7	T8
GĐ0-1 Chuẩn bị, nền tảng, hợp đồng								
GĐ2 Tài khoản, cửa hàng, tìm kiếm, giỏ, đơn, VNPAY								
GĐ3 Bản đồ, túi, kho, đơn của tôi, quầy								
GĐ4 Hoàn tiền, chi trả, đối soát, quản trị								
GĐ5 Test chéo, sửa lỗi, báo cáo								
Cổng kiểm tra (hợp cuối giai đoạn)								

3. Làm gì trước, làm gì sau: từng giai đoạn

GD0 · Chuẩn bị (tuần 1, ngày 1-2)

Ai	Làm theo thứ tự	Xong khi
Cả nhóm	1. Cài JDK 17+, Maven, Node 20+, MySQL 8.0.16+, Git, IntelliJ / VS Code, Postman. 2. Chạy <code>foodrescue_schema.sql</code> rồi <code>foodrescue_checks.sql</code> . 3. Đọc tài liệu dự án mục 2-5 và toàn bộ Quy ước chung. 4. Tạo tài khoản GitHub, gửi tên cho TV1.	Mỗi máy có <code>foodrescuedb</code> , câu C0 ra chệnh lệch 0, các câu C1-C12 không trả dòng nào
TV1	F1 tạo repo → F2 khung Spring Boot (chép <code>quy_uoc_chung/backend</code>) → F5 khung React (chép <code>quy_uoc_chung/frontend</code>) → push lên <code>develop</code> cuối ngày 2	Người khác <code>git clone</code> và chạy được cả hai phần
TV2	S0: làm component <code>LocationPicker</code> (ghim bản đồ) trong một project Vite nháp; phác giao diện đăng ký cửa hàng và quản lý túi	Bấm lên bản đồ ra toạ độ; có bản nháp giao diện
TV3	B0: class Java nháp gọi OSRM với 3 toạ độ; làm 6 ảnh danh mục (PNG nền trong) và 3 banner (Canva) theo mục 6.4 Quy ước chung	In được quảng đường và thời gian; có đủ ảnh
TV4	O0: đăng ký VNPAY Sandbox (email trả key có thể mất vài ngày); viết <code>VnPaySigner</code> + unit test trong project nháp	Đã gửi đăng ký; hàm ký cho đúng chữ ký với bộ dữ liệu mẫu

Cổng G0 (cuối ngày 2): ☐ `git clone` → `mvn spring-boot:run` chạy, `/swagger-ui.html` mở được, gọi `/api/orders` không có token trả 401 dạng `ApiError` · ☐ `npm run dev` thấy 3 khu, mỗi khu một trang "Đang xây dựng" · ☐ `MoneyTest pass` · ☐ mỗi người đã tạo nhánh đầu tiên và mở PR thử.

GD1 · Nền tảng dùng chung và hợp đồng API (tuần 1, ngày 3-5)

Ai	Làm theo thứ tự	Xong khi
TV1	F3 13 entity (ngày 3) → F4 bảo mật JWT + DevAuth (ngày 3-4) → F8 3 hàm dùng chung (ngày 4-5) → F6 lưu file, F7 Postman → H1 hợp đồng API của mình	Ứng dụng khởi động với <code>ddl-auto=validate</code> (tức entity khớp CSDL); gọi API bằng header <code>X-Dev-User-Id</code> được; 3 hàm dùng chung có test
TV2	Đưa <code>LocationPicker</code> vào repo → H2 hợp đồng API của mình → giao diện tĩnh trang đăng ký cửa hàng, trang quản lý túi (dữ liệu giả)	API của TV2 hiện trên Swagger; 2 trang tĩnh chạy trong repo
TV3	B14 bộ component giao diện dùng chung (header, thanh danh mục, thẻ túi, nhãn giá, nhãn trạng thái...) → trang chủ tĩnh theo mẫu → H3 hợp đồng API của mình	Trang chủ tĩnh giống mô phỏng mục 6.4 Quy ước chung; component nằm trong <code>shared/components</code>
TV4	H4 hợp đồng API của mình → thử tạo link thanh toán sandbox bằng thẻ test (nếu đã có key) → giao diện tĩnh trang giỏ hàng	API của TV4 hiện trên Swagger; trả tiền thử thành công trên sandbox

Cổng G1 (buổi họp cuối tuần 1, khoảng 90 phút): ☐ Swagger trên `develop` hiện đủ API ở mục 8 tài liệu dự án, mỗi API trả dữ liệu giả đúng định dạng · ☐ cả nhóm duyệt tên trường JSON theo từ điển thuật ngữ · ☐ entity khớp CSDL · ☐ `StockService` có test giữ hàng · ☐ trang chủ tĩnh đạt mẫu · ☐ TC25 (trigger chặn sửa sổ kho, nhật ký) đạt · ☐ TV1 gắn tag `v0.1`.

GD2 · Chức năng lõi (tuần 2-3)

Ai	Tuần 2	Tuần 3	Xong khi
TV1	A1 API tài khoản → A2 trang đăng nhập / đăng ký / hồ sơ / đổi mật khẩu	A5 admin đơn hàng và hoàn tiền (backend trước, giao diện sau)	Đăng nhập thật cho 4 vai trò, DevAuth không còn cần ở frontend
TV2	S1 API đăng ký cửa hàng → S2 trang đăng ký cửa hàng	S3 admin duyệt cửa hàng + đặt phí → S4 API tạo / sửa túi	Chủ quán đăng ký → admin duyệt → chủ quán tạo túi bằng API
TV3	B1 API danh sách túi + danh mục → B2 trang chủ nổi API thật	B3 tính khoảng cách → B4 định vị, "Vị trí nhận hàng" trên header → B12 admin danh mục	Trang chủ hiện túi thật, lọc danh mục, "2,4 km · 8 phút"
TV4	O12 API giỏ hàng → O1 API tạo đơn → O2 test đặt đồng thời	O3 tích hợp VNPay → O4 nút thêm vào giỏ, trang giỏ hàng, trang kết quả	Thêm giỏ → đặt → trả tiền sandbox → đơn "Chờ nhận hàng"

Cổng G2 (cuối tuần 3): chạy thật trên `develop`, không dùng dữ liệu giả: ☐ đăng ký GreenMart → admin duyệt với phí 10% · ☐ chủ quán tạo túi (Postman hoặc trang tạm) · ☐ khách mở trang chủ thấy túi kèm khoảng cách, lọc được danh mục · ☐ thêm 2 loại túi cùng quán vào giỏ → đặt → trả bằng thẻ test → đơn CONFIRMED + PAID · ☐ đạt **TC01, TC02, TC03, TC04, TC06, TC09, TC21, TC22** · ☐ sau đó `foodrescue_checks.sql` sạch · ☐ tag `v0.3`.

GD3 · Nhận hàng, bản đồ, quản lý túi (tuần 4-5)

Ai	Tuần 4	Tuần 5	Xong khi
TV1	A5 (tiếp): lịch sử tiền của đơn, admin huỷ đơn, trang hoàn tiền	A9 trang nhật ký → hỗ trợ ghép luồng mua hàng, sửa lỗi merge	Admin xem và xử lý được đơn, hoàn tiền; xem được nhật ký
TV2	S5 trang quản lý túi → S6 huỷ túi	S7 hồ sơ cửa hàng → S8 thêm / bớt túi + trang sổ kho	Chủ quán tự quản lý túi, kho trên giao diện
TV3	B5 trang bản đồ → B6 trang chi tiết túi (gắn nút thêm vào giỏ)	B7 trang cửa hàng công khai → B11 dữ liệu demo	Khách tìm, xem bản đồ, xem chi tiết, thêm vào giỏ
TV4	O5 đơn của tôi + QR → O6 2 tác vụ tự động	O7 màn hình quầy → O8 giao hàng bằng mã	Nhân viên giao được đơn; đơn tự hết hạn / tự ghi không đến lấy

Cổng G3 (cuối tuần 5, luồng mua hàng trọn vẹn): ☐ chạy được bước 1-6 kịch bản demo mục 14.2 tài liệu dự án · ☐ đạt **TC05, TC07, TC08, TC17, TC18, TC23, TC26** · ☐ nạp `demo_data.sql` xong `foodrescue_checks.sql` vẫn sạch · ☐ tag `v0.5` lên `main`.

GD4 · Tiền và quản trị (tuần 6-7)

Ai	Tuần 6	Tuần 7	Xong khi
TV1	A6 chi trả theo kỳ (có hàm <code>createForStore</code> cho TV2)	A7 trang đối soát	Chi trả trọn vòng; đối soát chênh lệch 0
TV2	S9 tài khoản nhân viên → S13 admin quản lý người dùng	S10 doanh thu, sao kê, yêu cầu rút tiền	Cửa hàng thấy từng đồng của mình và rút được tiền
TV3	B8 đánh giá	B9 hoàn thiện giao diện điện thoại → B13 thống kê chung	Khách đánh giá được; admin có trang thống kê

Ai	Tuần 6	Tuần 7	Xong khi
TV4	O9 đơn của cửa hàng + huỷ đơn	O10 chạy lại toàn luồng đơn, sửa lỗi	Mọi đường đi của đơn hàng đều đúng

Cổng G4 (cuối tuần 7, đủ tính năng): ☐ đạt **TC10-TC16, TC19, TC24** · ☐ chạy đủ 26 test case một lượt, ghi kết quả vào bảng · ☐ trang đối soát và câu C0 cùng ra chênh lệch 0 · ☐ dừng thêm tính năng mới · ☐ tag `v0.9` .

GĐ5 · Hoàn thiện và bàn giao (tuần 8)

Ngày	Cả nhóm làm	Kết quả
Ngày 1	Chạy lại <code>foodrescue_schema.sql</code> cho sạch; test chéo theo cặp: TV1 test module TV4, TV2 test TV3, TV3 test TV2, TV4 test TV1; mỗi lỗi mở 1 Issue nhãn <code>bug</code> , ghi cách tái hiện	Danh sách lỗi có mức độ (cao: tiền, kho, phân quyền)
Ngày 2-3	Sửa lỗi mức cao trước; mỗi lần sửa phần tiền / kho / đơn thì chạy lại <code>foodrescue_checks.sql</code> ; chạy lại 26 test case	Không còn lỗi mức cao; C0-C12 sạch
Ngày 4	TV1 merge <code>develop</code> → <code>main</code> , tag <code>v1.0</code> , (tùy chọn) deploy; mỗi người viết phần báo cáo của module mình + chụp màn hình	Bản phát hành cuối; các phần báo cáo
Ngày 5	TV1 ghép báo cáo và slide; tập demo 2 lần theo mục 14.2 tài liệu dự án, bấm giờ; quay video dự phòng	Báo cáo, slide, video demo, bảng kết quả test

4. Thở việc chi tiết

BE backend · **FE** frontend · **BE+FE** cả hai. "N3" = ngày 3 của tuần 1. Tên nhánh Git: `feature/tv<số>-<mã>-<mô tả>`, vd `feature/tv4-03-vnpay`.

TV1 · Nền tảng, Tài khoản, Tiền

Mã	Tuần	Việc	Các bước làm	Cần trước	Kết quả bàn giao	Kiểm tra	Giờ
F1	1 · N1	Tạo repo	1. Tạo repo foodrescue gồm backend/, frontend/, database/, docs/. 2. Chép .editorconfig, .gitattributes, .github/ từ quy_uoc_chung; chép 2 file SQL vào database/. 3. Tạo nhánh develop; bảo vệ main, develop (bắt buộc PR + 1 duyệt). 4. Tạo nhánh shared-change, api, db, bug; bảng GitHub Projects 5 cột. 5. Mời 3 thành viên.	—	Repo có develop; mọi người clone được	Push thẳng lên develop bị từ chối	2
F2	1 · N1-2	BE Khung Spring Boot	1. Spring Initializr: Java 17, Boot 3.3.x, Web, Data JPA, Security, Validation, MySQL Driver, Lombok; thêm jwt, springdoc-openapi. 2. Chép quy_uoc_chung/backend (common/*, application.yml, MoneyTest). 3. Tạo application-local.example.yml (mật khẩu MySQL, JWT, VNPay), thêm bản thật vào .gitignore. 4. CORS cho localhost:5173. 5. API GET /api/ping công khai.	F1	mvn spring-boot:run nổi được foodrescuedb; Swagger mở được	MoneyTest pass; /api/ping trả 200	5
F5	1 · N1-2	FE Khung React	1. Vite + React; cài antd @ant-design/icons axios react-router-dom dayjs react-leaflet leaflet qrcode.react recharts. 2. Chép quy_uoc_chung/frontend; thêm font Be Vietnam Pro; bọc ConfigProvider theme={antdTheme} locale={viVN}. 3. 3 layout: Customer (header trắng), Store và Admin (menu trái). 4. app/routes.jsx gộp routes của 5 feature; tạo sẵn file routes.jsx, menu.js rỗng cho cả 4 người. 5. PrivateRoute roles=[...], lưu đăng nhập trong localStorage. 6. Vite proxy /api sang 8080.	F1	npm run dev thấy 3 khu; npm run build chạy được	Mở /admin khi chưa đăng nhập thì về /login	8
F3	1 · N3	BE 13 entity	1. Viết entity cho 13 bảng theo "Ghi chú JPA" (tài liệu dự án mục 10): @IdClass cho cart_items, enum dạng STRING, tiền BigDecimal, @ManyToOne(fetch = LAZY), @Immutable cho StockMovement, AuditLog. 2. Repository cơ bản cho từng entity. 3. Chạy ứng dụng với ddl-auto=validate, sửa đến khi hết lỗi.	F2	13 entity + 13 repository; sau đó chỉ sửa qua PR [DB]	Ứng dụng khởi động không lỗi validate	6
F4	1 · N3-4	BE Bảo mật JWT	1. JwtService (tạo, đọc token 24 giờ), JwtAuthFilter. 2. Quy tắc URL theo khu vực (Quy ước chung mục 5.1). 3. CurrentUser trả {id, role, storeId, mustChangePassword}. 4. Trả 401 / 403 dạng ApiError. 5. DevAuth (profile dev): header X-Dev-User-Id giả lập người dùng. 6. Tài khoản mustChangePassword chỉ gọi được API đổi mật khẩu.	F3	Mọi người gọi API có quyền bằng DevAuth	Không token → 401; X-Dev-User-Id: 8 gọi /api/admin/.. → 403; 1 → 200	8

Mã	Tuần	Việc	Các bước làm	Cần trước	Kết quả bàn giao	Kiểm tra	Giờ
F8	1 · N4-5	BE 3 hàm dùng chung	1. StockService : reserve(batchId, qty, orderId) = UPDATE có điều kiện + ghi ORDERED, trả false nếu thiếu; release(...) ghi RETURNED; adjust(batchId, delta, reason) cho LISTED / ADDED / REMOVED; tự đổi AVAILABLE ⇔ SOLD_OUT. 2. AuditService.log(action, entityType, entityId, amount, detail). 3. RefundService.request(order, reason) : đơn CANCELLED + REFUND_PENDING, trả kho, tạo refunds, ghi log. 4. Test tích hợp trên CSDL mẫu.	F3	3 service để TV2, TV4 gọi	Test: 10 luồng giữ túi còn 1 → đúng 1 thành công; C1b sạch; TC25	5
F6, F7	1 · N5	BE Lưu file, Postman	1. FileStorageService.save(file) : chỉ jpg/png ≤ 5 MB, lưu /uploads, trả URL; phục vụ /uploads/**. 2. Postman collection + biến baseUrl, token, mỗi module một thư mục, lưu ở docs/postman.	F2	Upload được ảnh; collection chung	Upload file .exe đổi đuôi .jpg bị từ chối	5
H1	1 · N4-5	BE Hợp đồng API	Viết DTO (record) + controller trả dữ liệu giả cho /api/auth/login register, /api/me, /api/admin/orders refunds payouts finance audit-logs, theo Quy ước chung mục 5.5. Mở PR nhãn api.	F4	API hiện trên Swagger	Duyệt ở cổng G1	3
A1	2	BE API tài khoản	1. POST /api/auth/register (BCrypt, EMAIL_EXISTS). 2. POST /api/auth/login : chặn LOCKED, trả token + mustChangePassword + user. 3. GET/PUT /api/me. 4. PUT /api/me/password : kiểm tra mật khẩu cũ, tắt must_change_password.	F4	11 tài khoản mẫu đăng nhập được (mật khẩu 123456)	Postman thư mục Auth; một phần TC24	6
A2	2	FE Trang tài khoản	1. /login, /register : form theo Quy ước chung, lỗi hiện dưới ô nhập. 2. Sau đăng nhập về đúng khu theo vai trò. 3. /change-password bắt buộc khi có mật khẩu tạm. 4. /profile, nút đăng xuất trên 3 layout.	A1	Cả nhóm bỏ DevAuth ở frontend, dùng đăng nhập thật	Đăng nhập 4 vai trò vào đúng khu	6
A5	3-4	BE+FE Admin đơn hàng, hoàn tiền	1. GET /api/admin/orders lọc trạng thái, tìm mã; chi tiết kèm lịch sử tiền (thanh toán, hoàn, kỳ chi trả, nhật ký). 2. POST /api/admin/orders/{id}/cancel gọi RefundService; chặn đơn COMPLETED / NO_SHOW. 3. GET /api/admin/refunds, POST ../{id}/complete bắt buộc gatewayRef. 4. Trang /admin/orders, /admin/refunds.	F8, seed	Admin xử lý được đơn và hoàn tiền	TC10 (phần admin), TC15; C6 sạch	10
A9	5	BE+FE Trang nhật ký	1. GET /api/admin/audit-logs lọc hành động, loại, người, ngày, phân trang. 2. Trang /admin/audit-logs; bấm dòng để mở đơn / kỳ chi trả / cửa hàng.	F8	Thấy toàn bộ thao tác tiền và quản trị	Các thao tác ở G2-G3 đều có dòng log; C12 sạch	3
A6	6	BE+FE Chi trả theo kỳ	1. unpaid-summary, preview?storeId&from&to. 2. POST /api/admin/payouts : 1 transaction, gắn đơn bằng UPDATE ... WHERE payout_id IS NULL, chụp tài khoản ngân hàng, PAYOUT_PENDING_EXISTS. 3. /paid bắt buộc transferRef; /cancel gỡ đơn. 4. Tách PayoutService.createForStore() cho TV2. 5. Trang /admin/payouts.	seed	Admin chi trả trọn vòng; TV2 dùng được hàm	TC13, TC14; C7, C8, C9 sạch	10

Mã	Tuần	Việc	Các bước làm	Cần trước	Kết quả bàn giao	Kiểm tra	Giờ
A7	7	BE+FE Đối soát	1. GET <code>/api/admin/finance/reconciliation</code> : 7 khoản tiền theo mục 5.7 tài liệu dự án + chênh lệch + bảng theo cửa hàng. 2. Trang <code>/admin/finance</code> : chênh lệch khác 0 thì tô đỏ.	A6	Trang đối soát	TC16; số liệu trùng câu C0	6
A8	8	Phát hành, báo cáo	1. Merge <code>develop</code> → <code>main</code> , tag <code>v1.0</code> . 2. (Tuỳ chọn) deploy frontend Vercel, backend Render, MySQL cloud. 3. Ghép báo cáo, slide.	tất cả	Bản cuối, báo cáo, slide	Tập demo 2 lần	8
—	1-8	Việc nhóm trưởng	Chủ trì họp đầu / cuối tuần và các cổng; review PR của TV2; theo dõi bảng Projects; gắn tag ở mỗi cổng; duyệt mọi PR <code>shared-change</code> .	—	Biên bản họp mỗi tuần	—	8

TV2 · Cửa hàng, Túi, Kho

Mã	Tuần	Việc	Các bước làm	Cần trước	Kết quả bàn giao	Kiểm tra	Giờ
S0	1 · N1-3	FE Ghim bản đồ, giao diện tĩnh	1. <code>LocationPicker</code> (react-leaflet): bấm hoặc kéo ghim → trả <code>{lat, lng}</code> , chặn toạ độ ngoài Việt Nam. 2. Giao diện tĩnh trang đăng ký cửa hàng (3 bước) và trang quản lý túi, dùng component của TV3.	—	Component ghim + 2 trang tĩnh	Ghim ra đúng toạ độ	5
H2	1 · N4-5	BE Hợp đồng API	DTO + controller dữ liệu giả cho <code>/api/auth/register-store</code> , <code>/api/admin/stores*</code> , <code>/api/admin/users*</code> , <code>/api/store/batches*</code> , <code>quantity</code> , <code>stock-movements</code> , <code>profile</code> , <code>staff</code> , <code>revenue</code> , <code>statement</code> , <code>payouts</code> .	F4	API hiện trên Swagger	Duyệt ở G1	3
S1	2	BE API đăng ký cửa hàng	1. POST <code>/api/auth/register-store</code> (multipart): tạo user <code>STORE_OWNER</code> + store <code>PENDING</code> trong 1 transaction; lưu ảnh giấy phép qua <code>FileStorageService</code> . 2. Kiểm tra: email trùng, giờ đóng > giờ mở, toạ độ trong Việt Nam, đủ tài khoản ngân hàng. 3. Chủ quán chưa duyệt đăng nhập thì <code>storeStatus = PENDING</code> .	F4, F6	Đăng ký bằng Postman ra cửa hàng <code>PENDING</code>	Đăng ký lần 2 cùng email → <code>EMAIL_EXISTS</code>	5
S2	2	FE Trang đăng ký cửa hàng	1. <code>/store/register</code> : Steps tài khoản → cửa hàng + ghim → giấy phép + ngân hàng. 2. Xong thì báo "Chờ admin duyệt". 3. Chủ quán <code>PENDING</code> đăng nhập thấy trang "Đang chờ duyệt".	S1	Đăng ký được trên giao diện	Thử với dữ liệu thiếu, sai	6
S3	3	BE+FE Admin duyệt cửa hàng	1. GET <code>/api/admin/stores?status</code> ; <code>approve</code> (kèm tỉ lệ phí, mặc định 15%), <code>reject</code> (bắt buộc lý do), <code>lock</code> , <code>unlock</code> ; PUT <code>.../commission</code> . 2. Ghi log từng thao tác. 3. Trang <code>/admin/stores</code> : tab theo trạng thái, xem ảnh giấy phép, bản đồ vị trí.	F8, seed	Duyệt GreenMart với phí 10%	Nhật ký có <code>STORE_APPROVED</code>	8
S4	3	BE API túi	1. GET/POST/PUT <code>/api/store/batches</code> (multipart có ảnh), chỉ túi của cửa hàng mình. 2. Kiểm tra giá $\leq 60\%$ (<code>Money.isValidRescuePrice</code>), khung giờ, cửa hàng đã duyệt; túi đã có đơn thì không sửa giá / khung giờ. 3. Tạo túi gọi <code>StockService.adjust(LISTED)</code> .	F8	Tạo, sửa túi bằng API	TC09; C1b sạch	7
S5	4	FE Trang quản lý túi	1. <code>/store/batches</code> : bảng túi lọc trạng thái, <code>StatusTag</code> , <code>PriceTag</code> . 2. Form tạo / sửa: danh mục, ảnh, giá, số lượng, khung giờ. 3. Nút "Đăng lại" (chép túi cũ sang hôm nay).	S4	Chủ quán quản lý túi trên giao diện	Tạo túi mới hiện ngay trên trang chủ	7

Mã	Tuần	Việc	Các bước làm	Cần trước	Kết quả bàn giao	Kiểm tra	Giờ
S6	4	BE Huỷ túi	POST /api/store/batches/{id}/cancel trong 1 transaction: túi CANCELLED; đơn CONFIRMED có túi đó → RefundService.request(BATCH_CANCELLED) ; đơn PENDING_PAYMENT → CANCELLED + UNPAID, trả kho; ghi log BATCH_CANCELLED .	F8	Huỷ túi an toàn	TC26; C1c, C6 sạch	4
S7	5	BE+FE Hồ sơ cửa hàng	GET/PUT /api/store/profile ; trang sửa thông tin, ảnh, ghim lại vị trí, giờ mở cửa, tài khoản ngân hàng.	S0	Chủ quán sửa được hồ sơ	Đổi vị trí → khoảng cách ở trang chủ đổi theo	5
S8	5	BE+FE Thêm / bớt túi, sổ kho	1. PATCH .../quantity {delta} (nhân viên dùng được) gọi StockService.adjust (ADDED / REMOVED). 2. GET .../stock-movements . 3. Nút +/- trên bảng túi; trang sổ kho từng túi.	S4, F8	Thêm / bớt túi, xem sổ kho	TC18, TC23; C1, C1b sạch	6
S9	6	BE+FE Nhân viên	1. POST /api/store/staff : tạo STORE_STAFF với store_id của chủ, mật khẩu tạm, must_change_password = TRUE . 2. Khoá / mở khoá. 3. Trang /store/staff .	A1	Chủ quán quản lý nhân viên	Nhân viên mới đăng nhập bị bắt đổi mật khẩu, chỉ thấy menu quầy + túi	6
S13	6	BE+FE Admin người dùng	1. GET /api/admin/users lọc vai trò, tìm email; khoá / mở khoá (không khoá chính mình). 2. POST .../reset-password trả mật khẩu tạm 1 lần. 3. Ghi log. 4. Trang /admin/users .	F8	Admin quản lý tài khoản	TC24	6
S10	7	BE+FE Doanh thu, sao kê, rút tiền	1. GET /api/store/revenue (theo ngày). 2. GET /api/store/statement (từng đơn, kỳ chi trả / chưa chi). 3. GET /api/store/payouts . 4. POST /api/store/payouts/request gọi PayoutService.createForStore() . 5. Trang /store/revenue : thẻ số liệu, biểu đồ, bảng sao kê, nút "Yêu cầu rút tiền".	A6	Cửa hàng xem và rút được tiền	TC19; số liệu trùng câu C13	12
S11, S12	8	Test chéo, báo cáo	Test module TV3 (TC06, TC12 và giao diện khách); sửa lỗi của mình; viết phần báo cáo cửa hàng, túi, kho, CSDL.	—	Danh sách lỗi; phần báo cáo	—	10

TV3 · Khách hàng, Bản đồ, Giao diện chung

Mã	Tuần	Việc	Các bước làm	Cần trước	Kết quả bàn giao	Kiểm tra	Giờ
B0	1 · N1-2	Thử OSRM, làm ảnh	1. Class Java nháp gọi OSRM Table API, in mét / giây (nhớ thứ tự kinh độ, vĩ độ). 2. 6 ảnh danh mục PNG nền trong (đặt tên theo image_url trong SQL) + 3 banner (Canva) theo Quy ước chung mục 6.4.	—	Kết quả OSRM; bộ ảnh trong frontend/src/assets	So quãng đường với Google Maps	5

Mã	Tuần	Việc	Các bước làm	Cần trước	Kết quả bàn giao	Kiểm tra	Giờ
B14	1 · N3-5	FE Bộ component dùng chung	1. Trong <code>shared/components</code> : <code>SiteHeader</code> (logo, "Vị trí nhận hàng", ô tìm kiếm, Đăng nhập / Đăng ký, chỗ icon giỏ), <code>CategoryStrip</code> , <code>BatchCard</code> , <code>PriceTag</code> , <code>MoneyText</code> , <code>StatusTag</code> , <code>DistanceText</code> , <code>PickupWindow</code> , <code>PageHeader</code> , <code>StatCard</code> , <code>EmptyState</code> , <code>ConfirmAction</code> . 2. Chỉ dùng <code>theme.js</code> , <code>status.js</code> , <code>format.js</code> . 3. Trang chủ tĩnh với dữ liệu giả.	F5	Bộ component cho cả nhóm; trang chủ tĩnh giống mô phỏng	TV1 duyệt PR (nhấn <code>shared-change</code>)	6
H3	1 · N4-5	BE Hợp đồng API	DTO + controller dữ liệu giả cho <code>/api/categories</code> , <code>/api/batches</code> , <code>/api/batches/{id}</code> , <code>/api/stores/{id}</code> (+ reviews), <code>/api/orders/{id}/review</code> , <code>/api/admin/categories</code> , <code>/api/admin/dashboard</code> .	F4	API hiện trên Swagger	Duyệt ở G1	3
B1	2	BE API danh sách túi	1. GET <code>/api/categories</code> theo <code>sort_order</code> . 2. GET <code>/api/batches</code> : chỉ túi AVAILABLE, chưa hết giờ, cửa cửa hàng APPROVED; lọc danh mục, từ khoá; sắp xếp rẻ nhất / sắp hết giờ; <code>PageResponse</code> .	F3	API trả túi thật	Túi của cửa hàng PENDING không hiện	6
B2	2	FE Trang chủ	1. Nối <code>CategoryStrip</code> , lưới <code>BatchCard</code> với API. 2. Tab "Danh sách / Bản đồ", chọn bán kính, sắp xếp, "Xem thêm". 3. Trạng thái đang tải (<code>Skeleton</code>), trạng thái rỗng.	B1, B14	Trang chủ chạy dữ liệu thật	Lọc danh mục đúng	6
B3	3	BE Tính khoảng cách	1. Lọc cửa hàng bằng <code>ST_Distance_Sphere</code> trong bán kính (≤ 30). 2. Gọi OSRM 1 lần, timeout 2 giây. 3. Lỗi thì dùng chim bay $\times 1,3$ và <code>estimated = true</code> . 4. Sắp xếp "gần nhất". 5. Unit test trường hợp OSRM lỗi.	B1	Mọi thẻ túi có khoảng cách	TC06	9
B4	3	FE Vị trí người dùng	1. Hook <code>useGeolocation</code> (lỗi thì dùng trung tâm TP.HCM), lưu <code>localStorage</code> . 2. "Vị trí nhận hàng" trên <code>SiteHeader</code> : hiện địa chỉ, bấm để chọn lại trên bản đồ.	B3	Khoảng cách theo đúng vị trí người dùng	Từ chối quyền vị trí vẫn dùng được	4
B12	3	BE+FE Admin danh mục	CRUD <code>/api/admin/categories</code> có ảnh và thứ tự; không xóa được danh mục đang có túi; trang <code>/admin/categories</code> .	F6	Admin quản lý danh mục	Đổi thứ tự → thanh danh mục đổi theo	5
B5	4	FE Trang bản đồ	<code>/map</code> : vị trí tôi + vòng bán kính, marker cửa hàng, popup (tên, số túi, khoảng cách, link), nút chọn vị trí khác.	B3	Tìm túi trên bản đồ	Marker khớp danh sách	7
B6	4	BE+FE Chi tiết túi	1. GET <code>/api/batches/{id}?lat&lng</code> . 2. Trang <code>/batches/:id</code> : ảnh, mô tả, <code>PriceTag</code> , còn lại, khung giờ, bản đồ nhỏ, khoảng cách, đánh giá; chọn số lượng + <code><AddToCartButton></code> của TV4.	B1, O4	Thêm vào giỏ từ trang chi tiết	TC21 trên giao diện	6
B7	5	BE+FE Trang cửa hàng	GET <code>/api/stores/{id}</code> , <code>/reviews</code> ; trang thông tin, túi đang bán, đánh giá.	seed	Trang cửa hàng công khai	Cửa hàng bị khoá trả 404	5
B11	5	Dữ liệu demo	<code>database/demo_data.sql</code> : khoảng 15 cửa hàng APPROVED rải TP.HCM + 30 túi có khung giờ tối nay, mỗi túi kèm 1 dòng sổ kho LISTED cho khớp số lượng.	—	Bản đồ, danh sách đầy túi để demo	Chạy xong <code>foodrescue_checks.sql</code> vẫn sạch	4

Mã	Tuần	Việc	Các bước làm	Cần trước	Kết quả bàn giao	Kiểm tra	Giờ
B8	6	BE+FE Đánh giá	1. POST /api/orders/{id}/review : chỉ đơn COMPLETED của mình, 1 lần (REVIEW_NOT_ALLOWED). 2. <ReviewButton> (modal chấm sao) giao cho TV4. 3. Điểm trung bình hiện trên thẻ túi, trang cửa hàng.	seed	Khách đánh giá được	TC12; C10 sạch	7
B9	7	FE Hoàn thiện giao diện	Kiểm tra mọi trang khách ở 375px; trạng thái rõ ràng, lỗi; directionsUrl trong shared/utills/maps.js cho TV4; favicon, tiêu đề trang.	—	Giao diện khách dùng tốt trên điện thoại	Buổi review giao diện cuối tuần 7	5
B13	7	BE+FE Thống kê chung	GET /api/admin/dashboard?from&to : tổng đơn, GMV, túi đã cứu, số cửa hàng, số khách; trang /admin có StatCard + biểu đồ.	seed	Trang tổng quan admin	Số liệu khớp câu C13	7
B10	8	Test chéo, báo cáo	Test module TV2 (TC09, TC18, TC23, TC26); sửa lỗi của mình; viết phần báo cáo bản đồ, khoảng cách, giao diện.	—	Danh sách lỗi; phần báo cáo	—	5

TV4 · Giỏ hàng, Đơn, Thanh toán, Giao hàng

Mã	Tuần	Việc	Các bước làm	Cần trước	Kết quả bàn giao	Kiểm tra	Giờ
O0	1 · N1-3	BE Chuẩn bị VNPay	1. Đăng ký merchant sandbox, nhận vnp_TmnCode , vnp_HashSecret , gửi nhóm qua tin nhắn riêng. 2. VnPaySigner : sắp xếp tham số, URL-encode, HMAC-SHA512 + unit test. 3. Tạo 1 link thật, trả bằng thẻ NCB test.	—	Hàm ký đúng; đã trả tiền thử thành công	Link mở được trang VNPay	6
H4	1 · N4-5	BE Hợp đồng API	DTO + controller dữ liệu giả cho /api/cart* , /api/orders* , /api/payments/vnpay/* , /api/store/orders* , /api/store/pickup/* .	F4	API hiện trên Swagger	Duyệt ở G1	3
O12	2	BE API giỏ hàng	GET /api/cart (giá, số còn lại mới nhất, đánh dấu túi hết / hết giờ, khung giờ chung, canCheckout); thêm (1-5 mỗi loại), sửa, xoá, xoá hết; túi khác cửa hàng → CART_OTHER_STORE .	F3	Giỏ hàng chạy qua Postman	TC21 (backend)	4
O1	2	BE API tạo đơn	1. POST /api/orders lấy giỏ của khách. 2. Kiểm tra túi còn bán, cùng cửa hàng, có khung giờ chung. 3. Trong 1 transaction: StockService.reserve từng túi theo id tăng dần (thiếu thì rollback, báo túi nào); tạo orders + order_items ; mã đơn, mã nhận hàng; phí theo stores.commission_rate qua Money ; payments PENDING; xoá giỏ.	F8, O12	Tạo đơn nhiều túi đúng tiền, đúng kho	C1-C5 sạch sau khi tạo đơn	9
O2	2	BE Test đặt đồng thời	Test JUnit: 10 luồng cùng đặt túi còn 1; đơn 2 loại túi mà 1 loại thiếu.	O1	Test tự động chạy trong CI	TC01, TC22	4
O3	3	BE Tích hợp VNPay	1. Tạo paymentUrl (hết hạn 10 phút). 2. /api/payments/vnpay/return và /ipn cùng gọi confirm() : kiểm tra chữ ký, số tiền; chỉ xử lý 1 lần; 00 → CONFIRMED + PAID + log; mã khác → EXPIRED + trả kho. 3. Tiền về khi đơn đã EXPIRED: còn hàng thì giữ lại và CONFIRMED, hết thì RefundService(LATE_PAYMENT) .	O0, O1	Thanh toán thật trên sandbox	TC02, TC03, TC04, TC17	9

Mã	Tuần	Việc	Các bước làm	Cần trước	Kết quả bàn giao	Kiểm tra	Giờ
O4	3	FE Giỏ hàng, kết quả thanh toán	1. <AddToCartButton> giao TV3; hook useCart cho icon giỏ trên header. 2. Trang /cart : danh sách túi, sửa số lượng, tổng tiền, khung giờ chung, cảnh báo túi hết, nút "Đặt hàng". 3. /payment/result : gửi tham số về backend, hiện thành công + mã nhận hàng / thất bại.	O12, O3	Mua hàng trọn vẹn trên giao diện	Làm lại TC02, TC21 trên giao diện	8
O5	4	BE+FE Đơn của tôi	1. GET /api/orders , /{id} . 2. Trang /orders : tab theo trạng thái; đơn chờ nhận hiện mã + QR, khung giờ, "Chỉ đường"; đơn chưa trả có nút huỷ (POST .../cancel , trả kho); đơn bị huỷ hiện trạng thái hoàn tiền; chỗ gắn <ReviewButton> .	seed	Khách theo dõi được đơn	Huỷ đơn chưa trả → C1c sạch	8
O6	4	BE 2 tác vụ tự động	1. Mỗi phút: đơn PENDING_PAYMENT quá 15 phút → EXPIRED (UPDATE có điều kiện trạng thái) + StockService.release . 2. Mỗi 5 phút: túi quá giờ → EXPIRED; đơn CONFIRMED quá pickup_end + 30 phút → NO_SHOW + completed_at . 3. Ghi log.	O1	Đơn và túi tự đổi trạng thái	TC05; C1c, C5 sạch	5
O7	5	BE+FE Màn hình quầy	GET /api/store/orders?date=today (lọc theo storeId trong token); trang /store/counter : đơn hôm nay theo khung giờ, tự tải lại 15 giây, ô nhập mã lớn cho máy tính bảng.	seed	Nhân viên thấy đơn mới	Đơn mới hiện sau ≤ 15 giây	6
O8	5	BE+FE Giao hàng bằng mã	1. GET /api/store/pickup/{code} : đúng cửa hàng, CONFIRMED, trong khung giờ (sớm 15 phút, trễ 30 phút). 2. POST .../complete : COMPLETED, handled_by , completed_at , log. 3. Modal hiện tên khách + danh sách túi, nút "Đã giao".	O7	Giao hàng tại quầy	TC07, TC08; C11 sạch	5
O9	6	BE+FE Đơn của cửa hàng, huỷ đơn	/store/orders lọc ngày, trạng thái; chi tiết đơn các túi; chủ quán huỷ đơn (bắt buộc lý do) → RefundService.request(STORE_CANCELLED) .	F8	Cửa hàng xử lý đơn hết hàng	TC10 (phần cửa hàng)	6
O10	7	Kiểm tra toàn luồng đơn	Chạy TC01-TC05, TC07, TC08, TC17, TC21, TC22 trên giao diện; sửa lỗi; nếu đã deploy: thử IPN thật.	—	Luồng đơn không lỗi	Bảng kết quả test	6
O11	8	Test chéo, báo cáo	Test module TV1 (TC10, TC13-TC16, TC19, TC24); viết phần báo cáo giờ hàng, thanh toán, chống bán vượt.	—	Danh sách lỗi; phần báo cáo	—	7

5. Phân chia thư mục và điểm nổi

5.1 Ai sở hữu thư mục nào

Người	Backend <code>backend/src/main/java/com/foodrescue/</code>	Frontend <code>frontend/src/</code>	Khác
TV1	<code>common/</code> (config, security, entity, exception, storage) · <code>auth/</code> · <code>common/stock</code> , <code>common/audit</code> , <code>common/refund</code> · <code>admin/</code> (orders, refunds, payouts, reconciliation, audit-logs)	<code>app/</code> · <code>shared/</code> (axios, layouts, PrivateRoute, component chung) · <code>features/auth/</code> · <code>features/admin/</code>	<code>database/</code> · <code>pom.xml</code> · <code>package.json</code> · <code>README</code>
TV2	<code>store/</code> (register, approval, profile, staff, revenue, statement) · <code>batch/</code> · <code>adminuser/</code>	<code>features/store/</code> (trừ counter, orders) · <code>features/admin-stores/</code> · <code>features/admin-users/</code>	
TV3	<code>catalog/</code> (listing, distance, public store) · <code>review/</code> · <code>admindcatalog/</code> (categories, stats)	<code>features/catalog/</code> · <code>features/review/</code> · <code>features/admin-catalog/</code> · <code>shared/components/</code> (bộ giao diện chung, TV1 duyệt)	<code>database/demo_data.sql</code>
TV4	<code>cart/</code> · <code>order/</code> · <code>payment/</code> · <code>pickup/</code> · <code>scheduler/</code>	<code>features/order/</code> (giỏ hàng, kết quả thanh toán, đơn của tôi, quầy, đơn cửa hàng)	

Quy tắc repository: Spring Data cho phép nhiều repository cho cùng một entity. Mỗi người tự tạo repository trong package của mình với các câu truy vấn mình cần (ví dụ TV3 tạo `catalog/BatchSearchRepository` extends `JpaRepository<RescueBatch, Long>` chứa câu tìm túi theo vị trí). Riêng số lượng túi thì **chỉ** được đổi qua `StockService` của TV1 để sổ kho luôn khớp. Không ai phải sửa repository của người khác.

5.2 Điểm nổi giữa các module

Chỉ có vài chỗ module này dùng đồ của module khác. Mỗi chỗ đều có **phương án tạm** để người dùng không phải chờ:

Bên giao → bên nhận	Giao cái gì	Hạn	Hợp đồng (chốt tuần 1)	Trong lúc chờ
TV1 → cả nhóm	Khung BE/FE, entity, quy tắc URL, axios, layout, file routes/menu	Ngày 2	Cấu trúc thư mục mục 1.1; enum trạng thái đúng như SQL	Làm spike (OSRM, VNPay) và giao diện tĩnh
TV1 → cả nhóm	Đăng nhập JWT, <code>CurrentUser</code>	Tuần 2	<code>CurrentUser.get()</code> trả <code>{id, role, storeId}</code>	DevAuth: header <code>X-Dev-User-Id</code> (1 admin, 2 chủ An Phát, 6 nhân viên, 8 khách)
TV1 → TV2	<code>FileStorageService</code>	Tuần 1	<code>String save(MultipartFile file) → trả /uploads/xxx.jpg</code>	Lưu tạm đường dẫn cố định
TV1 → cả nhóm	<code>StockService</code> , <code>AuditService</code> , <code>RefundService</code>	Tuần 1	<code>stockService.reserve / release / adjust</code> · <code>auditService.log(action, entityType, entityId, amount, detail)</code> · <code>refundService.request(order, reason)</code>	Tạm tự viết UPDATE số lượng (xóa khi có <code>StockService</code>); tạm bỏ qua ghi log
TV1 → TV2	<code>PayoutService.createForStore()</code>	Tuần 6	<code>Payout createForStore(Long storeId, LocalDate from, LocalDate to)</code> , ném lỗi nếu không có đơn hoặc đang có kỳ PENDING	Ấn nút "Yêu cầu rút tiền"

Bên giao → bên nhận	Giao cái gì	Hạn	Hợp đồng (chốt tuần 1)	Trong lúc chờ
TV4 → TV3	<AddToCartButton>	Tuần 3	<AddToCartButton batch={{id, storeId, rescuePrice, remainingQuantity}} quantity={n} /> trong features/order/components/	Nút giả hiện "Sắp có"
TV3 → TV4	<ReviewButton>	Tuần 6	<ReviewButton order={{id, status, reviewed}} onDone={fn} /> trong features/review/components/	Không hiện nút
TV3 → TV4	Util chỉ đường	Tuần 7	directionsUrl(lat, lng) trong shared/utls/maps.js (TV3 tạo file mới)	Tự viết link tạm
TV4 ↔ TV1, TV2	Trả lại số lượng khi hủy đơn	—	Mỗi người tự viết câu UPDATE trong repository của mình: remaining_quantity = remaining_quantity + :qty, status về AVAILABLE nếu đang SOLD_OUT và chưa hết giờ	—

5.3 Quy ước JSON chung (chi tiết đầy đủ: Quy ước chung mục 5)

```
// Túi (TV3 trả; TV4 dùng trong AddToCartButton)
{ "id": 3, "title": "Surprise Box Bánh Pháp",
  "imageUrl": "/uploads/b3.jpg", "categoryName": "Bánh ngọt",
  "originalPrice": 120000, "rescuePrice": 45000,
  "remainingQuantity": 14, "status": "AVAILABLE",
  "pickupStart": "2026-09-25T21:00:00", "pickupEnd": "2026-09-25T22:00:00",
  "store": { "id": 2, "name": "Sweet Paris", "address": "...",
    "latitude": 10.781254, "longitude": 106.698942, "ratingAvg": 0 },
  "distanceMeters": 2350, "durationSeconds": 452, "estimated": false }
```

```
// Đơn (TV4 trả; TV1 admin và TV3 ReviewButton dùng)
{ "id": 4, "orderCode": "FR260925-0004", "itemCount": 2,
  "totalAmount": 87000,
  "commissionAmount": 13050, "storeEarning": 73950,
  "commissionRate": 15.00, "pickupCode": "H2R9W7",
  "status": "CONFIRMED", "payoutId": null,
  "paymentStatus": "PAID", "reviewed": false,
  "pickupStart": "2026-09-25T21:00:00", "pickupEnd": "2026-09-25T22:00:00",
  "items": [ { "batchId": 3, "title": "Surprise Box", "quantity": 1, "unitPrice":
    45000, "subtotal": 45000 },
    { "batchId": 4, "title": "Bánh Kem Lát", "quantity": 1, "unitPrice":
    42000, "subtotal": 42000 } ],
  "store": { "id": 2, "name": "...", "latitude": ..., "longitude": ... },
  "createdAt": "2026-09-25T10:20:00" }

// Lỗi (mọi API)
{ "code": "BATCH_SOLD_OUT", "message": "Túi vừa được giải cứu hết",
  "details": [], "timestamp": "2026-09-25T21:05:00", "path": "/api/orders" } // HTTP
409
```

Quy ước: JSON camelCase; tiền là số nguyên VNĐ; thời gian dạng ISO yyyy-MM-ddTHH:mm:ss giờ Việt Nam; danh sách phân trang trả { "content": [...], "totalElements": n, "page": 0, "size": 20 }.

6. Quy trình làm việc với Git

6.1 Cấu trúc repo và nhánh

```
foodrescue/                (1 repo GitHub, 4 người cùng push)
├── backend/                Spring Boot (mvn)
├── frontend/              React + Vite (npm)
├── database/
│   ├── foodrescue_schema.sql  CSDL + dữ liệu mẫu (TV1 giữ)
│   └── demo_data.sql         dữ liệu demo (TV3)
├── docs/                  tài liệu, postman/, ảnh chụp màn hình
└── README.md

main      ← chỉ nhận merge từ develop cuối mỗi mốc (luôn chạy được để demo)
develop   ← nhánh tích hợp; mọi PR đều vào đây
feature/tv2-batch-crud ← nhánh làm việc của từng người, mỗi công việc 1 nhánh
```

- Tên nhánh: `feature/tv<số>-<mã việc>-<mô tả>`, ví dụ `feature/tv4-03-vnpay`; sửa lỗi dùng `fix/tv3-map-marker`.
- Bảo vệ `main` và `develop` trên GitHub: không push trực tiếp, bắt buộc Pull Request + 1 người duyệt.
- Cặp review: **TV1 ↔ TV2**, **TV3 ↔ TV4**. Review trong vòng 24 giờ.
- Merge kiểu **Squash and merge** để lịch sử `develop` gọn (mỗi PR 1 commit). Xóa nhánh sau khi merge.

6.2 Quy ước commit

```
feat(order): tạo đơn và trừ kho có điều kiện
fix/catalog): sửa thứ tự tọa độ khi gọi OSRM
ui(store): form tạo túi dùng DatePicker
test(order): test 10 luồng đặt cùng lúc
docs: cập nhật README cách chạy
db: thêm cột image_url cho stores ← chỉ TV1, kèm thông báo nhóm
```

6.5 File dễ xung đột và quy tắc

File	Chủ	Quy tắc
database/foodrescue_schema.sql, các entity	TV1	Ai cần thêm cột: tạo Issue → TV1 sửa cả SQL + entity trong 1 PR tiêu đề [DB] → báo nhóm chạy lại script. Không tự sửa.
pom.xml, package.json	TV1	Thêm thư viện bằng một PR nhỏ riêng (chỉ file đó), merge ngay để người khác pull. Không trộn với PR tính năng.
SecurityConfig	TV1	Quy tắc theo tiền tố URL đã chốt ở tuần 1; đặt API đúng tiền tố thì không cần sửa file này.

6.3 Các bước mỗi lần làm một việc

```
# 1. Lấy code mới nhất
git checkout develop
git pull origin develop

# 2. Tạo nhánh cho việc mới
git checkout -b feature/tv3-B5-map-page

# 3. Code, commit nhỏ và thường xuyên
git add .
git commit -m "feat(catalog): trang bản đồ với marker cửa hàng"

# 4. Trước khi tạo PR: cập nhật theo develop
git fetch origin
git rebase origin/develop      # có xung đột thì sửa, git add, git rebase --continue
# (không quên rebase thì dùng: git merge origin/develop)

# 5. Đẩy lên và mở Pull Request vào develop trên GitHub
git push -u origin feature/tv3-B5-map-page

# 6. Sau khi PR được merge
git checkout develop && git pull
git branch -d feature/tv3-B5-map-page
```

6.4 Cuối mỗi công kiểm tra (tuần 1, 3, 5, 7, 8)

```
# TV1 thực hiện sau buổi demo nội bộ
git checkout main && git pull
git merge --no-ff origin/develop
git tag v0.5 -m "Chạy trọn luồng mua hàng"
git push origin main --tags
```

File	Chủ	Quy tắc
application.yml	TV1	Chỉ chứa cấu hình chung. Mật khẩu MySQL, key VNPay, JWT secret để trong application-local.yml (đã gitignore), mẫu ở application-local.example.yml .
Routes, menu React	Từng người	Mỗi feature có routes.jsx và menu.js riêng; app/routes.jsx chỉ import. Không sửa file của feature khác.
shared/ (component chung)	TV1	Được thêm file mới ; muốn sửa file có sẵn thì hỏi TV1 trước.
package-lock.json	—	Luôn commit. Xung đột thì lấy bản trên develop rồi chạy lại npm install .

7. Hợp nhóm, theo dõi tiến độ và Definition of Done

7.1 Nhịp làm việc mỗi tuần

Khi nào	Nội dung
Đầu tuần (20-30 phút, online)	Mỗi người nói việc tuần này (theo mã ở mục 3), vướng gì; kéo thẻ Trello sang "Tuần này".
Hàng ngày (nhắn nhóm)	Một dòng: hôm qua xong gì / hôm nay làm gì / có bị chặn không. Bị chặn quá 1 ngày thì báo ngay, không ngồi chờ.
Cuối tuần (45-60 phút)	Mỗi người demo phần mình trên nhánh <code>develop</code> (không demo trên nhánh riêng); ghi biên bản ngắn (phục vụ báo cáo môn); ở các mốc thì TV1 merge <code>develop</code> → <code>main</code> và gắn tag.

7.2 Bảng Trello / GitHub Projects

Các cột: **Backlog** → **Tuần này** → **Đang làm** → **Chờ review** → **Xong**. Mỗi thẻ ghi mã việc (F1, S4, O3...), tên người, link PR. Có thể tạo nhanh bằng GitHub Issues, gắn nhãn `tv1 ... tv4`, `be`, `fe`.

7.3 Một việc được coi là "Xong" khi

- API chạy đúng khi gọi bằng Postman, kể cả trường hợp lỗi (dữ liệu sai, không đủ quyền); request đã lưu vào collection chung.
- Giao diện gọi API thật (không còn dữ liệu giả), có trạng thái đang tải và thông báo lỗi; không lỗi đỏ trong Console.
- Dùng được trên màn hình điện thoại (Chrome DevTools, 375px).
- Đã rebase theo `develop`, build được (`mvn package`, `npm run build`).
- PR được người review duyệt và đã merge vào `develop`; thẻ Trello chuyển sang "Xong".

7.4 Khi bị trễ

- Luồng mua hàng (tuần 2-5) là ưu tiên số 1. TV4 trễ thì TV1 vào hỗ trợ ngay.
- Có thể cắt hoặc làm đơn giản nếu thiếu giờ: biểu đồ (B13, S10 → chỉ hiện bảng số), trang cửa hàng công khai (B7), dữ liệu demo (B11), QR (chỉ hiện mã chữ).
- Không được cắt: chống bán vượt, giỏ hàng, sổ kho, kiểm tra chữ ký VNPay, phân quyền, hoàn tiền, chi trả theo kỳ và đối soát.